Project Plan

Purpose. This document is the single top-level roadmap for **Radiation Effects Project 2**. It now replaces the separate "module 0 overview" plus the older project-plan split. It defines the physics scope, the repository map, the run-order workflow, the recommended build order, the dimensional assumptions, and the next expansion path.

Major planning update in this revision. Module 1 remains the Geant4 radiation-deposition module, while **Modules 2 through 6 are now planned as two-dimensional models**. The prior one-dimensional intermediate path is no longer the main target architecture. One-dimensional checks may still be used privately as verification subproblems, but the official module definitions and interfaces below are now **2D-first**.

1. Project objective

The project title is **Radiation-Induced Defect Evolution and Device Degradation in Silicon**. The central question is how a microscopic radiation interaction inside silicon becomes a macroscopic change in device behavior.

The physics chain is:

1. incident radiation deposits energy in silicon,
2. deposited energy seeds point defects and defect complexes,
3. those defects migrate, react, cluster, and anneal,
4. charged defects alter the internal electrostatic landscape,
5. the changed field and trap population alter carrier transport,
6. thermal evolution modifies rates and mobilities,
7. the coupled system produces measurable device degradation.

2. Module roadmap

No.	Module	Purpose
1	Geant4 radiation energy deposition in silicon	Compute where, when, and how much energy is deposited in silicon by the incident radiation field. Export a deposition map that seeds later material-damage or defect-generation models.
2	Two-dimensional electrostatics with defect-dependent space charge	Solve the two-dimensional electrostatic problem for a damaged silicon structure. Convert dopants and charged defects into electric potential, electric field, and depletion-structure changes.
3	Two-dimensional defect evolution: diffusion, reaction, migration, and annealing	Evolve vacancies, interstitials, complexes, and effective defect species in two spatial dimensions as functions of time and temperature.

4	Two-dimensional thermal transport coupled to defect evolution	Solve the temperature field in the device and use it to update diffusion coefficients, reaction rates, annealing rates, and other temperature-dependent physics.
5	Two-dimensional drift-diffusion carrier transport with defect-assisted recombination	Predict how the damaged device transports electrons and holes under the altered field and trap landscape.
6	Coupled two-dimensional multiphysics degradation model	Couple Modules 2–5 into a self-consistent simulator for defect evolution, field evolution, thermal evolution, and carrier-transport degradation.
7	Multiscale extrapolation and scalable prediction	Develop methods for making predictions on much larger spatial, temporal, or statistical scales than the direct high-fidelity simulation can afford. This module covers controlled approximations, quantified error bars, and statistical upscaling.

3. Core architecture decision: 2D for Modules 2–6

3.1. Why the move to 2D matters

The original one-dimensional path was numerically attractive, but it suppresses several effects that become important once the damage pattern is spatially structured:

- lateral spreading of defect populations,
- lateral field bending and edge effects,
- nonuniform temperature fields near localized energy deposition,
- current crowding and local carrier-flow rerouting,
- spatial mismatch between Geant4 deposition structure and downstream device solvers.

A two-dimensional architecture is therefore the more faithful bridge between a realistic Geant4 source pattern and later electrostatic, thermal, and transport calculations.

3.2. Practical interpretation

"2D" here means the main solvers are planned on an (x, y) mesh. For time-dependent modules the fields become functions such as

$$C(x, y, t), \qquad \phi(x, y, t), \qquad T(x, y, t), \qquad n(x, y, t), \qquad p(x, y, t).$$

Module 1 can still export either a true 2D deposition map directly or a set of event/deposition products that are binned into a 2D source map before passing to MATLAB.

3.3. What still stays simple

The move to 2D does *not* mean every solver should start maximally complex. The first versions should still keep:

- simple rectangular domains,
- structured grids,
- small defect networks,

- simple boundary conditions,
- weak one-way coupling before self-consistent feedback.

4. Run-order workflow for the project

The build order and the run order are related but not identical. The **run order** below is the logical flow of a coupled study.

Step 1. Module 1: Geant4 radiation transport
Run the Geant4 silicon geometry and score deposited energy, hit coordinates, optional timing information, and any derived damage-source products.

↓

Step 2. Geant4-to-MATLAB handoff
Bin or interpolate the Geant4 output into a two-dimensional deposition map or initial defect-source map on a MATLAB-side grid.

↓

Step 3. Module 3: defect evolution
Evolve the two-dimensional defect state under diffusion, reactions, clustering, dissociation, annealing, and optional source terms.

↓

Step 4. Module 4: thermal field
Solve or update the two-dimensional temperature field and use it to modify defect and transport coefficients.

↓

Step 5. Module 2: electrostatics
Build the net space-charge density from dopants and charged defects, then solve for electric potential and electric field.

↓

Step 6. Module 5: carrier transport
Solve the two-dimensional electron and hole transport problem with field-driven drift, concentration-driven diffusion, and defect-assisted recombination or trapping.

↓

Step 7. Module 6: coupled iteration
Iterate Modules 2–5 until the defect state, temperature field, electric field, and carrier state are mutually consistent to a chosen tolerance.

↓

Step 8. Module 7: scalable prediction
Apply reduced-order, approximation, or statistical-upscaling methods to extend the prediction to larger devices, larger domains, longer times, or rarer event statistics.

Important distinction. Module 4 does not always have to be solved before Module 2 in every simplified study. In the full intended workflow, however, the thermal field should be treated as available before the tightly coupled 2D solve, because temperature changes diffusion, annealing, mobility, and recombination parameters.

5. Recommended coding order

The most stable implementation order is still physics-driven rather than strictly numerical:

1. **Module 1:** Geant4 geometry, scoring, and export format.
2. **Module 3:** two-dimensional defect evolution on a structured grid.
3. **Module 2:** two-dimensional electrostatics with prescribed charged defects.
4. **Module 4:** two-dimensional thermal solver plus temperature-dependent coefficients.
5. **Module 5:** two-dimensional drift-diffusion with prescribed electrostatic field and defect/trap fields.
6. **Module 6:** weak coupling, then stronger feedback and iterative convergence.
7. **Module 7:** scalable-prediction methods after at least one credible high-fidelity coupled workflow exists.

This order starts with the damage-state evolution because later modules require the defect field. Electrostatics should be debugged before full carrier transport. Module 7 should come *after* at least one trustworthy baseline exists, because any approximation or scaling strategy needs a reference solution against which its error can be measured.

6. File-map overview

The current repository already has the top-level structure shown below, with separate documentation, Geant4, and MATLAB areas. The repository presently includes top-level overview and plan files, a `docs/` folder with `implementation_notes` and `physics_notes`, a `geant4/` folder, and a `matlab/` folder. The MATLAB area already contains top-level module drivers plus source subfolders such as `config`, `constants`, `coupling`, `defects`, `electrostatics`, `thermal`, `transport`, and `utils`.[1]

6.1. Top-level map

Path	Role
`project_plan.tex/.pdf`	Single living roadmap document: project objective, module definitions, run order, file map, interfaces, and roadmap updates.
`README.md`	Public repository summary and high-level entry point.
`docs/physics_notes/`	One large physics note per module, with derivations, symbol definitions, assumptions, and physical interpretation.
`docs/implementation_notes/`	Module-by-module implementation specifications, solver notes, and interface notes.
`geant4/`	Geant4-side setup, macros, and example outputs for Module 1.
`matlab/`	MATLAB drivers, source code, tests, output files, and cross-module interfaces for Modules 2–7.

6.2. Current MATLAB map plus planned 2D emphasis

Path	Role in the updated 2D plan

[1] Observed from the public repository structure.

`matlab/main_module2_electrostatics.m`	Top-level entry for the two-dimensional electrostatic solve.
`matlab/main_module3_defect_evolution.m`	Top-level entry for the two-dimensional defect evolution solve.
`matlab/main_module4_thermal.m`	Top-level entry for the two-dimensional thermal solve.
`matlab/main_module5_drift_diffusion.m`	Top-level entry for the two-dimensional carrier-transport solve.
`matlab/main_module6_multiphysics.m`	Top-level coupled driver that orchestrates Modules 2–5.
`matlab/src/config/`	Parameter files, grid settings, boundary-condition flags, material data, and run controls.
`matlab/src/defects/`	Two-dimensional diffusion-reaction operators, source-map importers, defect network logic, annealing models, and diagnostics.
`matlab/src/electrostatics/`	Poisson operators, charge assembly, boundary conditions, and field postprocessing in two dimensions.
`matlab/src/thermal/`	Two-dimensional heat equation assembly, thermal material properties, and thermal postprocessing.
`matlab/src/transport/`	Drift-diffusion operators, recombination models, mobility models, flux assembly, and current extraction.
`matlab/src/coupling/`	Inter-module interfaces, staggered-iteration logic, relaxation methods, and convergence checks.
`matlab/src/utils/`	Shared helpers, interpolation, file I/O, plotting, and diagnostics.
`matlab/tests/`	Verification problems and regression tests, including limiting-case checks.
`matlab/output/`	Saved run products, snapshots, coupling interfaces, and histories.

6.3. Planned Geant4 map

Path	Role
`geant4/macros/`	Run macros for geometry, beam/source definition, physics-list configuration, scoring, and batch studies.
`geant4/output_example/`	Example deposition outputs and exported files that illustrate the intended Geant4-to-MATLAB handoff.
`geant4/README_geant4.txt`	Instructions for compiling, running, and interpreting the Geant4 deposition workflow.

7. Module interfaces in the new 2D plan

7.1. Module 1 to Module 3 handoff

The first strong interface should now be a **2D deposition or defect-source map**. A minimal handoff object should contain:

- `x`: one-dimensional coordinate array in meters,
- `y`: one-dimensional coordinate array in meters,
- `Edep2D`: deposited-energy density field in $\mathrm{J\,m^{-3}}$,

- optional `tBins` and time-resolved source information,
- metadata describing bin width, device geometry, radiation case, and units.

A clean first file target is

`matlab/input/module1_to_module3_2d.mat`

so that the MATLAB side is not forced to parse raw Geant4 text every time.

7.2. Module 3 to Module 2 handoff

After a defect-evolution run, MATLAB should save

`matlab/output/module3_to_module2_2d.mat`

containing at minimum:

- `x`, `y`,
- `Cfinal(:,:,i)` for defect species or effective defect fields,
- `chargedDefectDensity2D` in m^{-3},
- `trapDensity2D` for later transport use,
- time-history diagnostics for provenance.

7.3. Module 2 and Module 4 to Module 5 handoff

Carrier transport should read:

- electric potential $\phi(x, y)$,
- electric field components $E_x(x, y), E_y(x, y)$,
- temperature $T(x, y)$,
- trap or recombination parameters derived from the defect state.

8. Verification strategy under the 2D architecture

The jump to 2D increases realism, but it also increases the number of ways a solver can fail. Each module therefore needs at least one low-complexity verification path.

Module	Verification logic
1	Compare deposition trends against known silicon behavior and verify that the exported 2D binning is internally consistent.
2	Use manufactured or symmetric charge configurations whose 2D potential and field patterns are physically obvious; verify sign conventions and boundary-condition behavior.
3	Check pure diffusion, pure annealing, and symmetry-preserving source cases in 2D. Total inventory should behave correctly under the chosen boundaries.
4	Recover known 2D heat-conduction behavior on simple rectangular domains with simple source terms and boundary conditions.
5	Verify defect-free transport first, then turn on simple trap-assisted recombination. Check that zero-field or symmetric cases behave sensibly.

6	Confirm that decoupled or weakly coupled limits reduce to standalone solutions. Track iterative residuals and relaxation behavior.
7	Every scalable-prediction method must be benchmarked against a higher-fidelity reference. The benchmark should report both speedup and error metrics.

9. Module 7: multiscale extrapolation and scalable prediction

9.1. Why Module 7 is needed

A direct two-dimensional coupled simulation can become expensive when one tries to predict:

- much larger device areas,
- much longer annealing times,
- many radiation realizations,
- rare-event or tail-probability behavior,
- large parameter sweeps over temperature, bias, dose, or geometry.

That is the motivation for a distinct roadmap item focused on scaling the predictive capability, not merely speeding up one code path.

9.2. Recommended name

A stronger title than "scaling up methods of the simulation" is:

Module 7. Multiscale Extrapolation and Scalable Prediction

This name is better because it covers both controlled approximation and statistical scaling, without implying that the only objective is raw performance.

9.3. Option A: controlled reduced-fidelity approximation

This branch intentionally simplifies the physics or numerics to gain speed. Examples include:

- coarser spatial meshes,
- simplified defect networks,
- operator splitting that neglects weak feedback terms in some iterations,
- quasi-steady assumptions for the fastest-relaxing field,
- reduced-order models built from high-fidelity snapshots,
- approximate Green-function or homogenized closures for selected subproblems.

The key requirement is not merely to run faster, but to **quantify what fidelity is lost**. For that reason, Module 7 should define an error metric relative to a higher-fidelity reference.

9.3.1. Suggested error-bar framework

For any approximate model, report at least:

$$\varepsilon_Q = \frac{\|Q_{\text{approx}} - Q_{\text{ref}}\|}{\|Q_{\text{ref}}\|},$$

where Q is a chosen quantity of interest, such as peak defect density, leakage current, charge-collection efficiency, depletion width, or a spatial field map. In addition, report a **speedup factor**

$$S = \frac{t_{\mathrm{ref}}}{t_{\mathrm{approx}}}.$$

A useful project-level trade plot is then ε_Q versus S, rather than speed alone.

For spatial fields one may also use weighted metrics such as

$$\varepsilon_w = \frac{\left(\int w(\mathbf{r})\,|Q_{\mathrm{approx}}(\mathbf{r}) - Q_{\mathrm{ref}}(\mathbf{r})|^2\,\mathrm{d}A\right)^{1/2}}{\left(\int w(\mathbf{r})\,|Q_{\mathrm{ref}}(\mathbf{r})|^2\,\mathrm{d}A\right)^{1/2}},$$

where the weight $w(\mathbf{r})$ emphasizes physically important regions such as junctions, depletion edges, or high-field regions.

9.4. Option B: statistical approximation and upscaling

This branch keeps the local physics model but uses statistics to predict larger-scale behavior. Examples include:

- using many smaller simulated tiles to infer a larger effective device response,
- propagating event statistics rather than simulating every event explicitly,
- surrogate distributions for defect-source fields,
- rare-event estimators or importance-weighted sampling,
- uncertainty propagation from source statistics to device-level degradation metrics.

This is especially relevant when the true goal is a prediction for a much larger area, a longer exposure, or a population-level reliability statement.

9.5. A third path worth keeping in view

There is a third useful category beyond the two branches above:

Hybrid multiresolution coupling

In this approach, only the physically critical region is simulated at high fidelity, while surrounding regions are treated with a cheaper closure or coarser model. Examples include local mesh refinement, embedded fine-grid windows around damage hot spots, or domain decomposition in which only one subregion uses the full coupled physics.

This third category often preserves more local fidelity than a globally reduced model, while still reducing cost significantly.

10. Immediate implications for the repository

Because of the change to 2D, the following top-level planning changes should now be treated as active:

1. top-level `project_plan.pdf` becomes the single maintained roadmap document,
2. the old module-0 overview is folded into this document conceptually,
3. all new implementation notes for Modules 2–6 should assume 2D unless explicitly labeled as a verification subcase,

4. future file names and interfaces should use a naming convention that makes dimension explicit when needed, such as `*_2d.m`, `*2D`, or metadata fields describing dimensionality,
5. cross-module MAT files should be designed around 2D arrays and explicit coordinate vectors.

11. When this project-plan document should be updated

This document should be revised whenever any of the following change:

- a module definition changes,
- the dimensionality assumption changes,
- a new top-level file or folder becomes part of the standard workflow,
- an inter-module file format changes,
- the run order changes,
- a major verification strategy changes,
- Module 7 gains a concrete benchmark or scaling method.

In practice, that means this file should be updated after any significant architectural change, not merely after cosmetic edits.

12. Near-term next steps after this revision

1. Update the repository so that this document is the only top-level planning PDF that must be maintained.
2. Rewrite the Module 3 implementation note around a 2D defect-evolution solver.
3. Define the first Geant4-to-MATLAB 2D handoff file and its units.
4. Update the Module 2, 4, and 5 implementation notes so their first solver architecture is 2D rather than 1D.
5. Decide the first benchmark quantity of interest for Module 7, so that future approximations can be measured against something physically meaningful.